

บทที่ 5

สรุปผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

ผู้จัดทำพัฒนาแอปพลิเคชัน Chatshop บนสมาร์ตโฟนด้วย Flutter และพัฒนาระบบบริการหลังบ้านด้วย Laravel 10 ร่วมกับฐานข้อมูล MySQL และ MongoDB เมื่อเทียบกับวัตถุประสงค์ในหัวข้อ 1.2 ได้ผลดังนี้

5.1.1 แอปพลิเคชันรวบรวมข้อความจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ไว้ในกล่องข้อความเดียว และเจ้าหน้าที่ตอบกลับข้อความรูปภาพ วิดีโอ และไฟล์เอกสารจากแอปพลิเคชันได้ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.2.1

5.1.2 ระบบส่งเหตุการณ์ผ่าน Laravel Reverb ทำให้แอปพลิเคชันแสดงข้อความใหม่ได้ทันที และส่งการแจ้งเตือนผ่าน Firebase Cloud Messaging เมื่อมีข้อความใหม่หรือแชทค้างตอบเกินกำหนด ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.2.2

5.1.3 เจ้าหน้าที่เปลี่ยนโหมดแชทบอท ดูผลวิเคราะห์บทสนทนา และมอบหมาย ปักหมุด จัดเก็บ และติดแท็กบทสนทนาได้จากแอปพลิเคชัน ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.2.3

5.1.4 เจ้าหน้าที่จัดการข้อมูลลูกค้าและสินค้า และเปิดคำสั่งซื้อที่ผูกกับบทสนทนาได้จากหน้าต่างสนทนา ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.2.4

5.1.5 แอปพลิเคชันเชื่อมต่อระบบหลังบ้านผ่าน REST API ประมาณ 99 เส้นทาง ยืนยันตัวตนด้วยโทเคนของ Laravel Sanctum และตรวจสิทธิ์ตามบทบาททุกคำขอ ซึ่งเป็นไปตามวัตถุประสงค์ข้อ 1.2.5

5.1.6 ผลการทดสอบด้วยชุดทดสอบอัตโนมัติในกลุ่มที่อยู่ในขอบเขตของโครงการ ระบบหลังบ้านผ่าน 575 จาก 577 กรณี และแอปพลิเคชันผ่าน 136 จาก 140 กรณี โดยกรณีที่ไม่ผ่านอยู่ในส่วนการตั้งค่าเท่านั้น

5.2 อภิปรายผล

จากการพัฒนาและทดสอบระบบ ผู้จัดทำมีข้อสังเกต ดังนี้

5.2.1 การแยกข้อมูลไว้ใน 2 ฐานข้อมูลตามลักษณะของข้อมูลเหมาะกับระบบนี้ ข้อมูลร้านค้า ทีม และคำสั่งซื้อที่ต้องการความถูกต้องอยู่ใน MySQL ส่วนข้อความที่เพิ่มขึ้นตลอดและมีหลายรูปแบบอยู่ใน MongoDB แต่ต้องตรวจสอบรหัสที่อ้างอิงข้ามฐานข้อมูลในโปรแกรมเอง

5.2.2 การตอบกลับ Webhook ทันทีแล้วประมวลผลในคิวงานช่วยให้แพลตฟอร์มไม่ส่ง Webhook ซ้ำ และการบันทึกรหัสข้อความขาเข้าที่ตอบไปแล้วทำให้ลูกค้าไม่ได้รับคำตอบซ้ำ แม้งานเดียวกันจะถูกประมวลผลหลายครั้ง

5.2.3 การแสดงผลการกระทำบนแอปพลิเคชันทันทีที่ก่อนได้รับคำตอบจากเครื่องแม่ข่าย ทำให้ผู้ใช้รู้สึกว่าแอปพลิเคชันตอบสนองเร็ว และย้อนกลับสถานะเดิมได้เมื่อการทำงานไม่สำเร็จ

5.2.4 การรอรอรวมข้อความ 6 วินาทีก่อนแชทบอทตอบ ทำให้แชทบอทตอบครั้งเดียวโดยเห็นคำถามทั้งหมด และการส่งต่อบทสนทนาให้เจ้าหน้าที่ช่วยลดความเสี่ยงที่แชทบอทจะตอบผิด

5.2.5 การค้นคืนเอกสารความรู้ด้วยการเทียบความคล้ายโคไซน์กับทุกส่วนย่อยทำงานได้ดีกับฐานความรู้ขนาดเล็กของร้านค้า แต่เวลาในการค้นคืนจะเพิ่มขึ้นตามจำนวนส่วนย่อย

5.3 ปัญหาและอุปสรรค

5.3.1 Meta อนุญาตให้ส่งข้อความอัตโนมัติได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดของลูกค้าเท่านั้น ผู้จัดทำจึงต้องให้ระบบส่งซ้ำพร้อมแท็ก HUMAN_AGENT เมื่อเจ้าหน้าที่ตอบเกินกรอบเวลา และจัดส่งข้อความอัตโนมัติในกรณีนี้

5.3.2 ไฟล์สื่อจาก LINE ไม่มี URL สาธารณะ ระบบจึงต้องดาวน์โหลดผ่าน API แล้วคัดลอกมาเก็บในระบบก่อนแสดงบนแอปพลิเคชัน

5.3.3 token ของเพจ Facebook หมดอายุหรือถูกยกเลิกได้ ระบบต้องตรวจจับข้อผิดพลาด ปิดการเชื่อมต่อช่องทางนั้น และแจ้งให้ร้านค้าเชื่อมต่อใหม่

5.3.4 การเชื่อมต่อเรียลไทม์หลุดได้เมื่อสัญญาณเครือข่ายไม่เสถียร แอปพลิเคชันจึงต้องดึงรายการบทสนทนาใหม่ทุกครั้งที่เชื่อมต่อกลับมาได้

5.3.5 ยังมีกรณีทดสอบที่ไม่ผ่าน 3 กรณีในส่วนการตั้งค่าธุรกิจและการตั้งค่าแชทบอทตามหัวข้อ 4.20

5.4 ข้อเสนอแนะและแนวทางในการพัฒนาต่อ

5.4.1 แก้ไขกรณีทดสอบที่ไม่ผ่านทั้ง 3 กรณี และเพิ่มการจำลอง flutter_secure_storage ในชุดทดสอบของแอปพลิเคชัน

5.4.2 เพิ่มการบันทึกและส่งข้อความเสียง การส่งสติ๊กเกอร์ และการคัดกรองบทสนทนาตามแท็กหรือสถานะในแอปพลิเคชัน

5.4.3 กำหนดเวลาหมดอายุของคำขอไปยังระบบหลังบ้าน และเก็บข้อมูลบางส่วนไว้บนอุปกรณ์ให้เปิดดูได้ขณะไม่มีสัญญาณเครือข่าย

5.4.4 เตรียมแอปพลิเคชันให้พร้อมเผยแพร่บนระบบปฏิบัติการ iOS

5.4.5 เพิ่มหน้าบรอดแคสต์และหน้าตั้งค่ากฎตอบกลับอัตโนมัติในแอปพลิเคชัน

5.4.6 เมื่อฐานความรู้มีขนาดใหญ่ขึ้น ควรเปลี่ยนไปใช้ดัชนีสำหรับค้นหาเวกเตอร์แทน
การเทียบกับทุกส่วนย่อย